

RHCSA EX200 — Resumo Dia 06/05

Scripts Shell · grep · sed · find · Variáveis de Ambiente

PROVA: 14/05

! Armadilhas do Dia

Certo	Errado
chmod +x sempre após criar o script	Esquecer chmod +x — script correto, mas eliminatório porque não roda
#!/bin/bash na linha 1, sem espaço antes do #	Shebang ausente ou mal posicionado — sistema tenta executar como binário
\$(id -u \$USER) — retorna só o número do UID	\${id \$USER} — retorna a linha inteira (uid=0(root) gid=...)
for NUM in "\$@" (com aspas)	for NUM in \$@ (sem aspas) — quebra argumentos com espaço
while read VAR < arquivo (redirecionamento)	cat arquivo while read VAR — pipe cria subshell, variáveis somem
comando > arquivo 2>&1 (ordem correta)	comando 2>&1 > arquivo — stderr ainda vai para o terminal
find ... -perm -4000 (pelo menos este bit)	find ... -perm 4000 (exige permissão EXATAMENTE 4000)

Tabela-Chave — Variáveis Especiais do Shell

Sintaxe	Significado
\$1 \$2 \$3	argumentos posicionais (1º, 2º, 3º)
\$@	todos os argumentos como lista
\$#	número de argumentos
\$()	substituição de comando
\$(())	aritmética inteira
-gt -lt -eq	comparadores numéricos: maior, menor, igual

BLOCO 1 — Estrutura Base de Todo Script

Todo script no EX200 segue exatamente a mesma estrutura. Internalize como um ritual: shebang → lógica → chmod +x → testar.

```
#!/bin/bash
# Linha 1: SEMPRE shebang #!/bin/bash

# Variáveis
NOME="valor"
RESULTADO=$(comando)           # capturar saída de comando
SOMA=$((NUM1 + NUM2))          # aritmética inteira

# Argumentos posicionais
$1 $2 $3                        # primeiro, segundo, terceiro argumento
"$@"                            # TODOS os argumentos como lista
$#                              # número de argumentos

# Criar, tornar executável e testar
vim /usr/local/bin/meu_script.sh
chmod +x /usr/local/bin/meu_script.sh
/usr/local/bin/meu_script.sh arg1 arg2
```

- `chmod +x` é eliminatório. Sem ele o avaliador não consegue executar o script para corrigir — perde o ponto mesmo com a lógica certa.
- Sem o shebang `#!/bin/bash` o sistema tenta executar como binário e falha. Sempre na linha 1, sem espaço antes do #.

BLOCO 2 — Argumentos: \$1, \$2, @\$

Exemplo completo (S1-Q11): flipargs.sh — inverter dois argumentos

```
#!/bin/bash
echo "$2 $1"

chmod +x /usr/local/bin/flipargs.sh

# Testar
/usr/local/bin/flipargs.sh hello world
# Output: world hello
```

- \$1 = primeiro argumento, \$2 = segundo. Inverter a ordem no echo é tudo que precisamos.

Exemplo completo (S1-Q30): checkfile.sh — verificar se arquivo existe

```
#!/bin/bash
if [ -f "$1" ]; then
    echo "File exists."
else
    echo "File missing."
fi

chmod +x /usr/local/bin/checkfile.sh

# Testar
/usr/local/bin/checkfile.sh /etc/passwd      # Output: File exists.
/usr/local/bin/checkfile.sh /etc/naoexiste   # Output: File missing.
```

Flags de teste — Referência:

Flag	Significado	Exemplo
-f	É arquivo regular	[-f "/etc/passwd"]
-d	É diretório	[-d "/tmp"]
-e	Existe (arquivo ou dir)	[-e "/caminho"]
-z	String vazia	[-z "\$VAR"]
-n	String não vazia	[-n "\$VAR"]

- Aspas em "\$1" evitam erros com nomes de arquivo com espaços. Sempre cite variáveis dentro de [].

BLOCO 3 — Loop for com Lista Fixa

Exemplo completo (S1-Q24): user_audit.sh — imprimir nome e UID de cada usuário

```
#!/bin/bash
for USER in root adm ftp
do
    USERID=$(id -u $USER)
    echo "User $USER has ID $USERID"
done

chmod +x /usr/local/bin/user_audit.sh

# Output:
# User root has ID 0
# User adm has ID 3
# User ftp has ID 14
```

- `$(id -u $USER)` retorna só o número do UID. `$(id $USER)` retorna `uid=0(root) gid=...` — texto completo que não é o pedido.
- `$()` é substituição de comando: executa o comando em subshell e captura a saída. Forma moderna de backticks — mais legível.

Variação (S3-Q26): user-list.sh — imprimir mensagem para cada usuário

```
#!/bin/bash
for USER in root bin daemon
do
    echo "Processing user: $USER"
done

chmod +x /usr/local/bin/user-list.sh
```

Template mental do for com lista:

```
for VARIÁVEL in item1 item2 item3
do
    # corpo do loop — $VARIÁVEL tem o item atual
done
```

BLOCO 4 — Loop for + \$@ + if — Somar Positivos

Destaque: este é o script mais complexo do exame — já caiu em dois simulados (S5-Q13 e S6-Q16). Combina: `$@` para número ilimitado de argumentos + `for` para iterar + `if` para filtrar + `$(())` para somar. Pratique até memorizar.

Exemplo completo (S5-Q13 / S6-Q16): sum.sh — somar apenas argumentos maiores que 0

```
#!/bin/bash
TOTAL=0
for NUM in "$@"
do
    if [ "$NUM" -gt 0 ]; then
        TOTAL=$((TOTAL + NUM))
    fi
done
echo "The sum is $TOTAL"

chmod +x /sum.sh

# Testar
/sum.sh 10 20 -5 30    # Output: The sum is 60 (ignora -5)
/sum.sh 10 20 -5      # Output: The sum is 30
```

Cada elemento deste script, explicado:

- `TOTAL=0` — inicializar o acumulador ANTES do loop; sem isso a aritmética pode falhar.
- `"$@"` — expande para todos os argumentos como itens separados; com aspas preserva espaços.
- `-gt 0` — greater than 0 — filtra negativos e zero.
- `=$((TOTAL + NUM))` — aritmética inteira; resultado atribuído à mesma variável.

Tabela de comparadores numéricos:

Operador	Significado	Exemplo
-gt	maior que	["\$N" -gt 0]
-lt	menor que	["\$N" -lt 10]
-eq	igual	["\$A" -eq "\$B"]
-ne	diferente	["\$A" -ne 0]
-ge	maior ou igual	["\$N" -ge 1]
-le	menor ou igual	["\$N" -le 100]

BLOCO 5 — Loop while — Ler Arquivo Linha por Linha

Exemplo completo (S4-Q22): process_lines.sh — processar arquivo com DATA NOME

```
# Primeiro criar o arquivo de teste
cat > /root/dates.txt << EOF
2000-05-01 Sam
2002-06-15 Sue
2004-12-29 Sarah
2005-05-10 Mike
EOF

#!/bin/bash
while read DATE NAME
do
    echo "Name: $NAME was born on $DATE"
done < /root/dates.txt

chmod +x /usr/local/bin/process_lines.sh

# Output:
# Name: Sam was born on 2000-05-01
# Name: Sue was born on 2002-06-15
# Name: Sarah was born on 2004-12-29
# Name: Mike was born on 2005-05-10
```

- done < /root/dates.txt redireciona o arquivo como entrada do loop. while read VAR1 VAR2 divide cada linha pelos espaços.
- NÃO use cat arquivo | while read (o pipe cria subshell — variáveis modificadas dentro não ficam visíveis fora). Use redirecionamento com <.

BLOCO 6 — read Interativo + Aritmética \$(())

Exemplo completo (S3-Q12): sum.sh interativo — pedir dois números ao usuário

```
#!/bin/bash
echo "Enter the first number"
read num1
echo "Enter the second number"
read num2
sum=$((num1 + num2))
echo "The result of addition=$sum"

chmod +x /usr/local/bin/sum.sh

# Execução interativa:
# /usr/local/bin/sum.sh
# Enter the first number
# 5
# Enter the second number
# 3
# The result of addition=8
```

Operações com \$(()):

Expressão	Operação
<code>\$((A + B))</code>	Soma
<code>\$((A - B))</code>	Subtração
<code>\$((A * B))</code>	Multiplicação
<code>\$((A / B))</code>	Divisão inteira
<code>\$((A % B))</code>	Módulo (resto)

- Leia o enunciado: 'pede ao usuário' → read interativo. 'aceita argumentos' → \$1 \$2. Confundir os dois = script errado.

BLOCO 7 — case + tr — Case-Insensitive

Exemplo completo (S2-Q9): yes-no.sh — resposta diferente para yes/no/outros

```
#!/bin/bash
# Converter para minúsculo para comparação case-insensitive
INPUT=$(echo "$1" | tr '[:upper:]' '[:lower:]')

case "$INPUT" in
    yes)
        echo "That is nice"
        ;;
    no)
        echo "I am sorry"
        ;;
    *)
        echo "Unknown argument"
        ;;
esac

chmod +x /usr/local/bin/yes-no.sh

# Testar
/usr/local/bin/yes-no.sh yes      # That is nice
/usr/local/bin/yes-no.sh YES      # That is nice (case-insensitive!)
/usr/local/bin/yes-no.sh no       # I am sorry
/usr/local/bin/yes-no.sh maybe    # Unknown argument
```

Anatomia do case:

- `tr '[:upper:]' '[:lower:]'` — converte tudo para minúsculas antes de comparar.
- `case "$INPUT" in` — abre o case; variável sempre entre aspas.
- `;;` — termina cada bloco — obrigatório, como um `break`. `*)` é o default, equivalente ao `else`. `esac` fecha o case ('case' ao contrário).
- case é mais limpo que `if/elif/else` para múltiplas opções do mesmo valor. Esquecer `;;` ou `esac` causa erro de sintaxe.

BLOCO 8 — Scripts com find — SUID e Auditoria

Exemplo completo (S3-Q28): find_suid.sh — listar arquivos SUID em /usr/bin

```
#!/bin/bash
find /usr/bin -user root -perm -4000

chmod +x /root/find_suid.sh
# Output: /usr/bin/sudo, /usr/bin/passwd, etc.
```

- `-perm -4000`: o hífen significa 'pelo menos este bit ativo'. Sem hífen (`-perm 4000`) exige permissão EXATAMENTE 4000.

Exemplo completo (S4-Q27): audit_suid.sh — copiar SUID < 10MiB para /root/audit_results/

```
#!/bin/bash
mkdir -p /root/audit_results
find /usr -type f -perm /u=s -size -10M -exec cp -a {} /root/audit_results/ \;

chmod +x /usr/local/bin/audit_suid.sh
ls /root/audit_results/
```

- `cp -a` preserva TODOS os atributos, inclusive o bit SUID. `-size -10M`: hífen = menor que; sem sinal = exato; + = maior que.

Variação (S5-Q29): audit_suid.sh — salvar lista de SUID < 5MiB em arquivo

```
#!/bin/bash
find /usr -type f -perm /4000 -size -5M > /root/suid_files.txt

chmod +x /usr/local/bin/audit_suid.sh
cat /root/suid_files.txt
```

Flags do find — Referência:

Flag	Significado	Exemplo
-perm -4000	SUID ativo (pelo menos)	find /usr/bin -perm -4000
-perm /u=s	SUID ativo (forma simbólica)	find /usr -perm /u=s
-perm /4000	SUID ativo (alternativa)	find /usr -perm /4000
-size -10M	Menor que 10MB	-size -5M (menor que 5MB)
-type f	Apenas arquivos regulares	-type d (diretórios)
-user root	Dono é root	-user alice
-exec CMD {} \;	Executar CMD em cada resultado	-exec cp -a {} /dest/ \;

BLOCO 9 — grep · sed · Redirecionamento

Referência rápida:

```
# GREP — busca
grep "texto" arquivo          # busca simples
grep -i "texto" arquivo       # case-insensitive
grep -r "texto" /diretorio    # recursivo
grep "^Host" arquivo          # linhas que COMEÇAM com "Host"
grep -v "texto" arquivo       # linhas que NÃO contêm
grep '-0[56]-' arquivo        # character class [56] = 5 ou 6
grep -E '-(05|06)-' arquivo   # regex estendida (OR)

# SED — substituição
sed 's/antigo/novo/' arquivo   # 1ª ocorrência por linha
sed 's/antigo/novo/g' arquivo  # TODAS (g = global)
sed 's/sam/Sam/g' letter > newletter # resultado em novo arquivo

# REDIRECIONAMENTO
comando > arquivo             # SOBRESCREVE
comando >> arquivo             # ACRESCENTA
comando 2> arquivo            # só stderr
comando > arq 2>&1             # stdout + stderr
comando &> arquivo             # stdout + stderr (atalho Bash)
```

Exemplo (S1-Q25): grep — linhas começando com 'Host' em sshd_config

```
# ^Host ancora no início da linha — exclui comentários automaticamente
grep -i '^Host' /etc/ssh/sshd_config > /root/ssh_hosts.txt
cat /root/ssh_hosts.txt
```

■ ^Host com ^ casa apenas linhas que COMEÇAM com Host. Comentários começam com # — não são incluídos.

Exemplo (S3-Q19): sed — substituir 'sam' por 'Sam' em todas as ocorrências

```
echo 'sam is here. hello sam. sam again.' > letter
sed 's/sam/Sam/g' letter > newletter
cat newletter      # Sam is here. hello Sam. Sam again.
cat letter         # sam is here. hello sam. sam again. (original intacto!)
```

■ Sem 'g': só a 1ª ocorrência por linha. sed não modifica o arquivo original — o > captura a saída em um novo arquivo.

Exemplo (S4-Q19): redirecionar stdout E stderr para o mesmo arquivo

```
ls /root /fake_directory > /var/tmp/ls_output.txt 2>&1
# OU forma curta Bash:
ls /root /fake_directory &> /var/tmp/ls_output.txt
cat /var/tmp/ls_output.txt # tem a listagem E o erro
```

■ Ordem importa: '> arquivo 2>&1' funciona. '2>&1 > arquivo' NÃO — o stderr vai para o terminal.

Exemplo (S4-Q21): grep com character class [56] — filtrar meses 05 e 06

```
cat > /root/dates.txt << EOF
2000-05-01 Sam
2002-06-15 Sue
2004-12-29 Sarah
2005-05-10 Mike
EOF

grep '-0[56]-' /root/dates.txt > /root/summer_birthdays.txt
# OU com regex estendida:
grep -E '-(05|06)-' /root/dates.txt > /root/summer_birthdays.txt
cat /root/summer_birthdays.txt # Sam (05), Sue (06), Mike (05)
```

■ [56] casa '5' ou '6'. Os hífens ao redor (-0[56]-) garantem que está no campo do mês — não confunde com o ano (2005).

BLOCO 10 — Variáveis de Ambiente

Exemplo completo (S3-Q24): export — variável na sessão atual e processos filhos

```
export EXAM_TYPE="RHCSA"

# Capturar saída de comando em variável
TODAY=$(date +%F) # ex: 2025-05-06

# Verificar em subshell (processo filho)
bash -c 'echo $EXAM_TYPE' # deve mostrar: RHCSA
echo $TODAY # data atual
```

■ export torna a variável disponível para processos filhos. Sem export, scripts rodados a partir desta sessão não a verão.

Exemplo completo (S4-Q20): /etc/profile.d/ — variável global para todos os usuários

```
vim /etc/profile.d/class_var.sh
# Conteúdo:
export CLASS="RHCSA"

# Verificar (simular login de outro usuário)
su - sarah
echo $CLASS # deve mostrar: RHCSA
```

■ Qualquer arquivo .sh em /etc/profile.d/ é executado no login de todos os usuários. Mais limpo e seguro que editar /etc/profile diretamente.

Exemplo completo (S5-Q24): /etc/environment — variável global inclusive via SSH

```
vim /etc/environment
# Conteúdo (sem 'export', sem lógica shell):
VAR="RHCSA Prac Five"

# Verificar (logout e login, ou:)
su -
echo $VAR # RHCSA Prac Five
```

■ Não é script shell — não usa export, não suporta lógica. Lido pelo PAM para todos os tipos de login, inclusive SSH.

Comparativo — quando usar cada abordagem:

Abordagem	Usa export?	Suporta lógica?	Funciona SSH?	Quando usar
export VAR=...	Sim	Sim	Não persiste	Sessão atual e filhos
/etc/profile.d/	Sim	Sim	Shell interativo	Variáveis/scripts globais com lógica
/etc/environment	Não	Não	Sim (PAM)	Variáveis estáticas para todos

Tabela de Referência Rápida — Comparadores Bash

Comparador	Significado	Exemplo
-eq	igual (números)	["\$A" -eq "\$B"]
-ne	diferente (números)	["\$A" -ne 0]
-gt	maior que	["\$NUM" -gt 0]
-lt	menor que	["\$NUM" -lt 10]
-ge	maior ou igual	["\$NUM" -ge 1]
-le	menor ou igual	["\$NUM" -le 100]
=	igual (strings)	["\$STR" = "sim"]
!=	diferente (strings)	["\$STR" != ""]
-f	é arquivo regular	[-f "/etc/passwd"]
-d	é diretório	[-d "/tmp"]
-z	string vazia	[-z "\$VAR"]
##	número de argumentos	["\$#" -ne 2]

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
Executar script	chmod +x arquivo	Esquecer chmod +x (não roda)
Argumentos com espaço	for NUM in "\$@" (com aspas)	for NUM in \$@ (sem aspas, quebra)
UID de usuário	\$(id -u \$USER)	\$(id \$USER) — retorna texto completo
Ler arquivo linha a linha	while read VAR < arquivo	cat arquivo while read VAR (subshell)
Redirecionar stdout+stderr	comando > arquivo 2>&1	comando 2>&1 > arquivo (ordem errada)
SUID exato vs pelo menos	-perm -4000	-perm 4000 (exige valor exato)
sed substituir todas ocorrências	sed 's/x/y/g'	sed 's/x/y/' (só a 1ª ocorrência)
Variável visível em subshell	export VAR=valor	VAR=valor (sem export, filho não vê)

✓ Checklist Final

Estrutura Base

- ☐ Todo script começa com #!/bin/bash e termina com chmod +x
- ☐ \$() para capturar saída de comando, \$(()) para aritmética

Argumentos

- ☐ flipargs.sh: echo "\$2 \$1" (inverter argumentos)
- ☐ checkfile.sh: if [-f "\$1"] then/else — aspas em \$1 obrigatórias

Loop for lista

- ☐ user_audit.sh: for USER in lista + USERID=\$(id -u \$USER)
- ☐ \$(id -u \$USER) retorna número — \$(id \$USER) retorna texto completo

for + \$@ + if

- ☐ sum.sh: TOTAL=0 antes do loop + for NUM in "\$@" + if ["\$NUM" -gt 0]
- ☐ "\$@" com aspas preserva argumentos — -gt = greater than

while + arquivo

- ☐ process_lines.sh: while read VAR1 VAR2 + done < arquivo

- ☐ NÃO usar pipe (cat | while) — subshell perde variáveis

read + aritmética

- ☐ sum.sh interativo: echo + read + \$((num1 + num2))
- ☐ Leia o enunciado: 'pede ao usuário' = read vs 'aceita argumentos' = \$1 \$2

case + tr

- ☐ yes-no.sh: tr '[:upper:]' '[:lower:]' antes do case
- ☐ Anatomia: case/in/opção/;;/*)/esac — esquecer ;; causa erro

find SUID

- ☐ find_suid.sh: find /usr/bin -user root -perm -4000
- ☐ -perm -4000 (hífen = pelo menos). -size -10M (hífen = menor que)
- ☐ -exec cp -a {} /dest/ \; preserva atributos incluindo SUID

grep • sed • redir

- ☐ grep -i '^Host' arquivo > saida.txt (^ ancora no início)
- ☐ sed 's/sam/Sam/g' arquivo > novo (g=todas, original intacto)
- ☐ > arquivo 2>&1 ou &> arquivo (ordem importa no 2>&1!)
- ☐ grep '[56]' para character class — grep -E '(05|06)' para OR

Variáveis Ambiente

- ☐ export VAR=valor para sessão atual e processos filhos
- ☐ /etc/profile.d/*.sh para variável global com suporte a lógica
- ☐ /etc/environment para variável global sem export (funciona SSH/PAM)